Parser mit ANTLR generieren

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Hello World

```
grammar Hello;

start : stmt* ;

stmt  : ID '=' expr ';' | expr ';' ;

expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;

atom  : ID | NUM ;

ID    : [a-z][a-zA-Z]* ;
NUM   : [0-9]+ ;
WS    : [ \t\n]+ -> skip ;
```

Konsole: Hello (grun, Parse-Tree)

Expressions und Vorrang (Operatoren)

```
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : ID ;
```

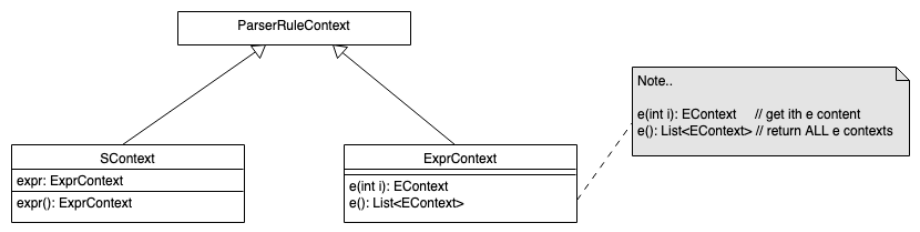
Expressions und Vorrang (Operatoren)

```
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : ID ;
```

```
expr  : expr '*' expr
      | expr '+' expr
      | ID
      ;
```

Kontext-Objekte für Parser-Regeln

```
s    : expr         {List<EContext> x = $expr.ctx.e();}
     ;
expr : e '*' e ;
```



Benannte Regel-Elemente oder Alternativen

```
stat  : 'return' value=e ';'    # Return
      | 'break' ';'             # Break
      ;
```

```
public static class StatContext extends ParserRuleContext { ... }
public static class ReturnContext extends StatContext {
    public EContext value;
    public EContext e() { ... }
}
public static class BreakContext extends StatContext { ... }
```

Arbeiten mit ANTLR-Listeners

```
expr : e1=expr '*' e2=expr      # MULT
     | e1=expr '+' e2=expr      # ADD
     | DIGIT                    # ZAHL
     ;
```

```
public static class MyListener extends calcBaseListener {
    public void exitMULT(calcParser.MULTContext ctx) {
        ...
    }
    public void exitADD(calcParser.ADDContext ctx) {
        ...
    }
    public void exitZAHL(calcParser.ZAHLContext ctx) {
        ...
    }
}
```

Arbeiten mit dem Visitor-Pattern

```
expr : e1=expr '*' e2=expr      # MULT
     | e1=expr '+' e2=expr      # ADD
     | DIGIT                    # ZAHL
     ;
```

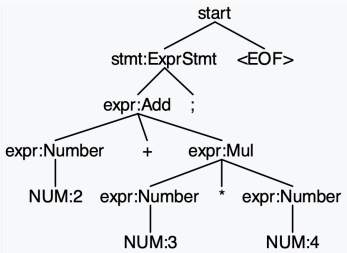
```
public static class MyVisitor extends calcBaseVisitor<Integer> {
    public Integer visitMULT(calcParser.MULTContext ctx) {
        return ...
    }
    public Integer visitADD(calcParser.ADDContext ctx) {
        return ...
    }
    public Integer visitZAHL(calcParser.ZAHLContext ctx) {
        return ...
    }
}
```

Arbeiten mit Pattern Matching

```
expr : e1=expr '*' e2=expr      # MULT
     | e1=expr '+' e2=expr      # ADD
     | DIGIT                    # ZAHL
     ;
```

```java
public static class PatternMatching {
    static Integer eval(calcParser.ExprContext e) {
        return switch (e) {
            case calcParser.MULTContext m -> eval(m.e1) * eval(m.e2);
            case calcParser.ADDContext a -> eval(a.e1) + eval(a.e2);
            case calcParser.ZAHLContext n -> Integer.parseInt(n.DIGIT().getText());
            default -> throw new IllegalStateException();
        };
    }
}
```

Vom Parse-Tree zum AST




```java
sealed interface Stmt permits Stmt.Assign, Stmt.ExprStmt {
  record Assign(String id, Expr value) implements Stmt {}
  record ExprStmt(Expr expr) implements Stmt {}
}

sealed interface Expr permits Expr.Mul, Expr.Add, Expr.Name, Expr.Number {
  record Mul(Expr lhs, Expr rhs) implements Expr {}
  record Add(Expr lhs, Expr rhs) implements Expr {}
  record Name(String id) implements Expr {}
  record Number(int value) implements Expr {}
}
```

Vom Parse-Tree zum AST (cnt.)

```java
static Stmt toAst(MyLangParser.StmtContext s) {
  return switch (s) {
    case MyLangParser.AssignContext a -> new Stmt.Assign(a.id.getText(), toAst(a.value));
    case MyLangParser.ExprStmtContext e -> new Stmt.ExprStmt(toAst(e.expr()));
    default -> throw new IllegalStateException();
  };
}

static Expr toAst(MyLangParser.ExprContext e) {
  return switch (e) {
    case MyLangParser.MulContext m -> new Expr.Mul(toAst(m.lhs), toAst(m.rhs));
    case MyLangParser.AddContext a -> new Expr.Add(toAst(a.lhs), toAst(a.rhs));
    case MyLangParser.NameContext n -> new Expr.Name(n.ID().getText());
    case MyLangParser.NumberContext n -> new Expr.Number(Integer.parseInt(n.NUM().getText()));
    default -> throw new IllegalStateException();
  };
}
```

Wrap-Up

Parser mit ANTLR generieren: Parser-Regeln werden mit **Kleinbuchstaben** geschrieben

- Regeln können Lexer- und Parser-Regeln "aufrufen"
- Regeln können Alternativen haben
- Bei Mehrdeutigkeit: Vorrang für erste Alternative
- ANTLR erlaubt direkte Links-Rekursion
- ANTLR erzeugt Parse-Tree
- Benannte Alternativen und Regel-Elemente
- Traversierung des Parse-Tree: Listener oder Visitoren oder *Pattern Matching*, Zugriff auf Kontextobjekte

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.